BUSINESS CASE:
TARGET SQL

Submitted by:
AJAY V MANOJ
ajayvm22@gmail.com

Submitted to:
SCALAR DSML
Submitted on:
28-07-2025

INTRODUCTION

This case study aims to uncover meaningful insights from a comprehensive e-commerce dataset that reflects the operations of **Target** in **Brazil**. Target is a globally recognized retail brand, known for delivering exceptional customer experiences and innovative services. By analysing data related to over **100,000 orders** placed between **2016 and 2018**, we aim to better understand customer behaviour, operational performance, and market dynamics in the Brazilian region.

The dataset spans across eight key dimensions including orders, customers, payments, product details, seller data, shipping information, customer reviews, and geolocation. This rich multi-table data enables a wide-ranging exploration — from analysing **order trends**, **customer distributions**, and **payment patterns** to evaluating **freight efficiency**, **delivery timelines**, and **customer satisfaction levels**.

The objective of this analysis is to:

- Perform **exploratory data analysis** to understand the structure and characteristics of the data.
- Identify trends and **seasonality** in customer orders.
- Evaluate the **economic impact** through payment values and shipping costs.
- Analyse **delivery timelines** and assess their alignment with estimated delivery dates.
- Study **payment behaviours**, including types and installment patterns.
- Provide **actionable recommendations** based on findings to help Target optimize its operations in Brazil.

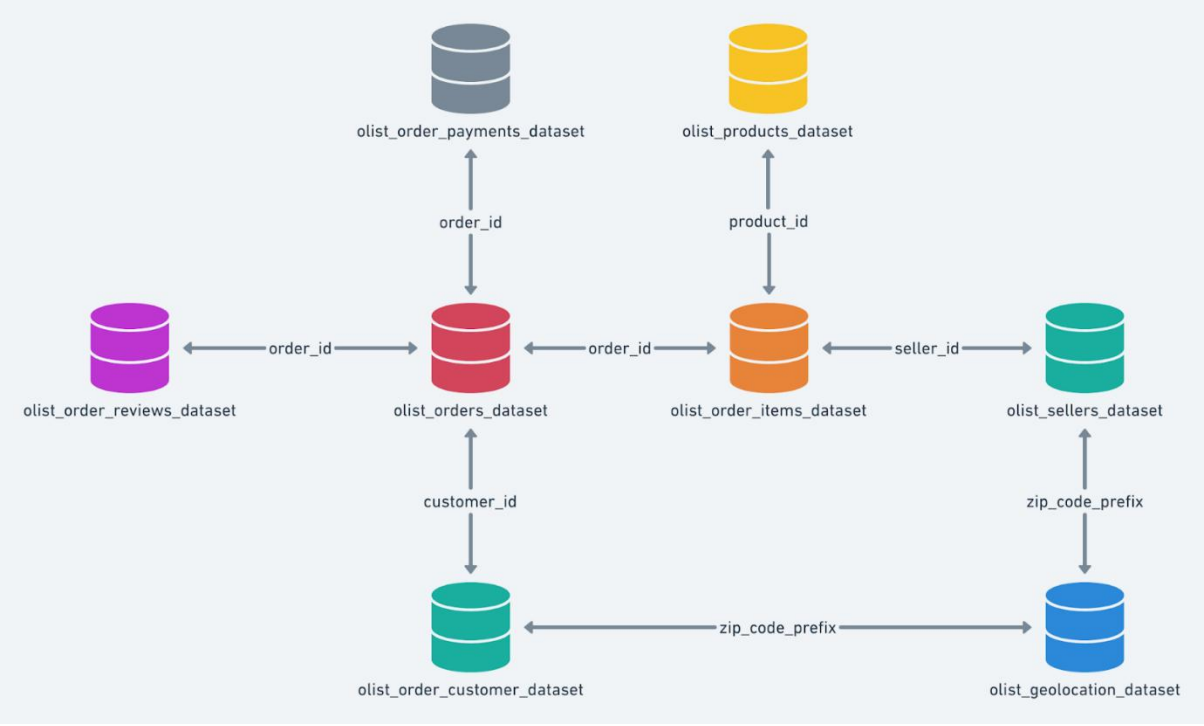
In addition to SQL-based analysis, this report will include **visual charts generated using Excel** wherever applicable to enhance the clarity and impact of insights derived.

Note: As per submission guidelines, screenshots will show only the first 10 rows of each query result. However, charts and visualizations are based on the full dataset to ensure meaningful insights.

DATABASE DESCRIPTION

The **Target** database is designed to capture and organize information related to an e-commerce platform. It includes detailed records of customer activity, product listings, order fulfilment, payment processing, and geolocation data. The structure of this database supports data-driven analysis across customer behaviour, seller performance, delivery timelines, and overall business metrics.

DATASET SCHEMA



TABLE'S AND TABLE DESCRIPTION

1. customers

Column Name	Data Type
customer_id	STRING

Column Name	Data Type
customer_unique_id	STRING
customer_zip_code_prefix	INTEGER
customer_city	STRING
customer_state	STRING

2. sellers

Column Name	Data Type
seller_id	STRING
seller_zip_code_prefix	INTEGER
seller_city	STRING
seller_state	STRING

3. order_items

Column Name	*Data Type*
order_id	STRING
order_item_id	INTEGER
product_id	STRING
seller_id	STRING
shipping_limit_date	TIMESTAMP
price	FLOAT
freight_value	FLOAT

4. geolocation

Column Name	*Data Type*

Column Name	*Data Type*
geolocation_zip_code_prefix	*INTEGER*
geolocation_lat	*FLOAT*
geolocation_lng	*FLOAT*
geolocation_city	*STRING*
geolocation_state	*STRING*

4. geolocation

Column Name	*Data Type*
geolocation_zip_code_prefix	INTEGER
geolocation_lat	FLOAT
geolocation_lng	FLOAT
geolocation_city	STRING
geolocation_state	STRING

5. payments

Column Name	Data Type
order_id	STRING
payment_sequential	INTEGER
payment_type	STRING
payment_installments	INTEGER
payment_value	FLOAT

6. orders

Column Name	*Data Type*

Column Name	*Data Type*
order_id	STRING
customer_id	STRING
order_status	STRING
order_purchase_timestamp	TIMESTAMP
order_approved_at	TIMESTAMP
order_delivered_carrier_date	TIMESTAMP
order_delivered_customer_date	TIMESTAMP
order_estimated_delivery_date	TIMESTAMP

7. reviews

Column Name	*Data Type*
review_id	STRING
order_id	STRING
review_score	INTEGER
review_comment_title	STRING
review_comment_message	STRING
review_creation_date	TIMESTAMP
review_answer_timestamp	TIMESTAMP

8. products

Column Name	*Data Type*
product_id	STRING
product_category_name	STRING
product_name_lenght	INTEGER

Column Name	***Data Type***
product_description_lenght	INTEGER
product_photos_qty	INTEGER
product_weight_g	INTEGER
product_length_cm	INTEGER
product_height_cm	INTEGER
product_width_cm	INTEGER

Problem Statements, Queries, and Insights

1. **Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:**

THE DATASET WAS UPLOADED TO GOOGLE BIG QUERY. ALL RELEVANT DETAILS ABOUT THE TABLES INCLUDING COLUMN NAMES, DATA TYPES, AND RELATIONSHIPS — HAVE BEEN DOCUMENTED IN THE DATABASE DESCRIPTION SECTION.

1.1 Data type of all columns in the "customers" table:

Query:

```
SELECT
  column_name,
  data_type
FROM
  TARGET.INFORMATION_SCHEMA.COLUMNS
WHERE
  table_name = 'customers';
```

output:

Row	column_name	data_type
1	customer_id	STRING
2	customer_unique_id	STRING
3	customer_zip_code_prefix	INT64
4	customer_city	STRING
5	customer_state	STRING

1.2 Get the time range between which the orders were placed:

Query:

```
SELECT
      MIN(order_purchase_timestamp) as Eariest_order_date_time,
      MAX(order_purchase_timestamp) as Latest_order_date_time
FROM
      `TARGET.orders`;
```

output :

Row	Earliest_order_date_time	Latest_order_date_time
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC

1.3 Count the Cities & States of customers who ordered during the given period:

Query:

```
SELECT
       COUNT(DISTINCT geolocation_city) AS city_count,
       COUNT(DISTINCT geolocation_state) AS state_count
FROM
       TARGET.geolocation;
```

output :

Row	city_count	state_count
1	8011	27

2. In-depth Exploration

2.1 Is there a growing trend in the no. of orders placed over the past years?

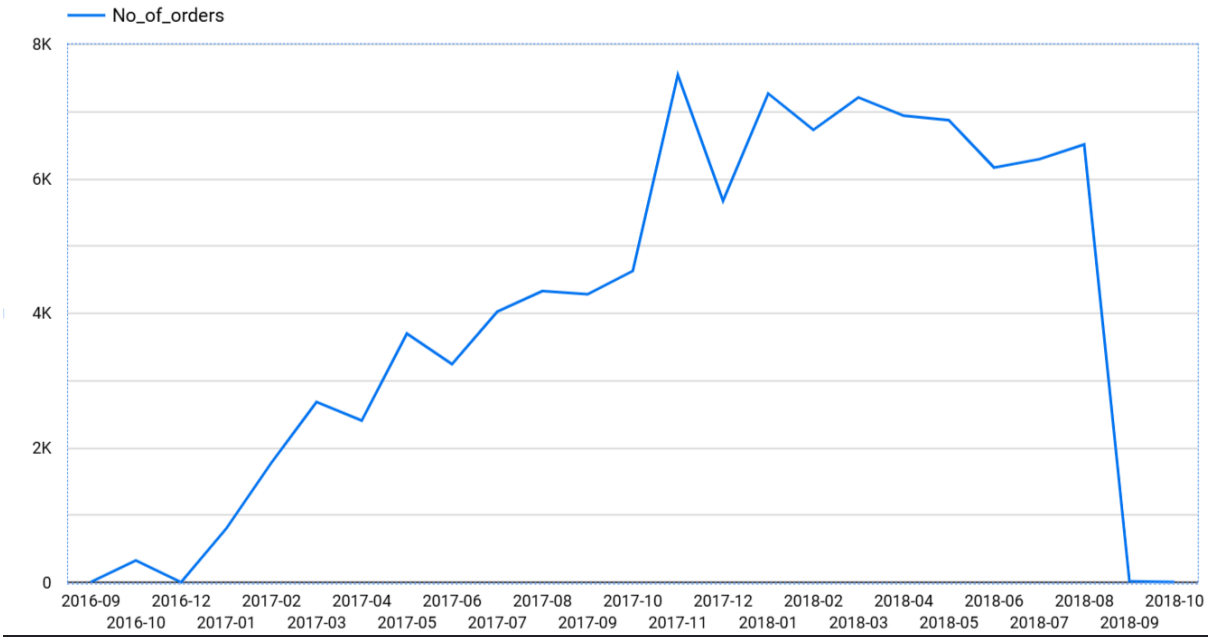
Query:

```
SELECT
      FORMAT_DATE('%Y-%m', order_purchase_timestamp) as Date,
      count(order_id) as No_of_orders
FROM
      TARGET.orders
GROUP BY 1
ORDER BY 1;


---- This query retrieves monthly order counts to analyse overall order
trends.
```

output:

Row	Date	No_of_orders
1	2016-09	4
2	2016-10	324
3	2016-12	1
4	2017-01	800
5	2017-02	1780
6	2017-03	2682
7	2017-04	2404
8	2017-05	3700
9	2017-06	3245
10	2017-07	4026

Insights and recommendations:



- Orders **grew steadily from late 2016 to 2017**, peaking at **7544 orders in November 2017** the **highest point** in the dataset.

- From **Nov 2017 to Aug 2018**, monthly orders **averaged around 6500** indicating a stable period with **lowest order count** in **Dec2017** at **5673**.

- **Sept and Oct 2018** saw a **steep decline** in orders, **dropping to 16 and 4** respectively likely due to incomplete data or operational changes.

Recommendation: *Consider investigating the sharp drop in Sept–Oct 2018 to identify if it's due to data issues or an actual operational concern. Ensuring complete and accurate data will improve the reliability of future trend analysis.*

2.2 Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

We already analysed the **month-on-month (MoM) order count** in the previous question. That analysis shows **no consistent monthly seasonality—order volumes fluctuate** but don't follow a repeatable monthly trend.

2.3 During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

- **0-6 hrs : Dawn**
- **7-12 hrs : Mornings**
- **13-18 hrs : Afternoon**
- **19-23 hrs : Night**

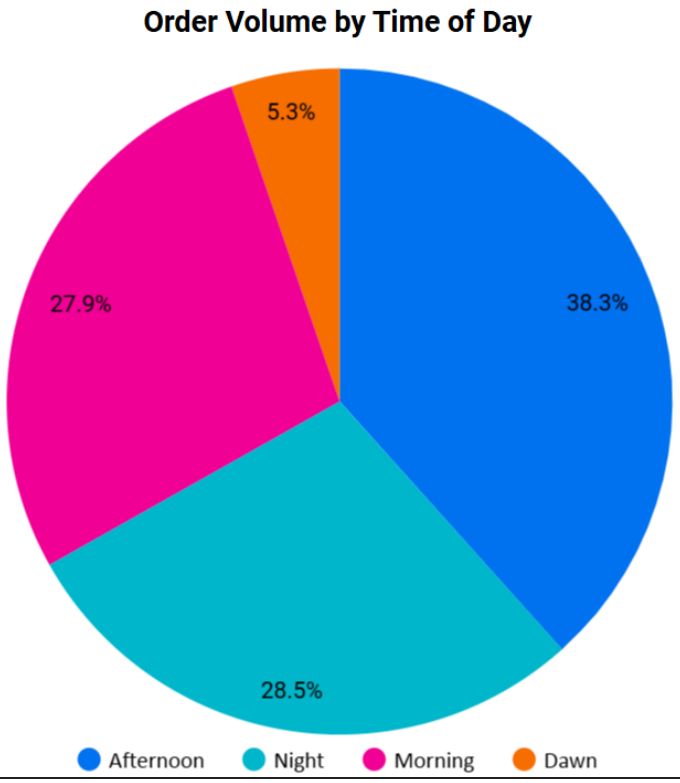
Query:

```
SELECT
      CASE WHEN EXTRACT(hour from order_purchase_timestamp) BETWEEN 0 AND 6
THEN 'Dawn'
           WHEN EXTRACT(hour from order_purchase_timestamp) BETWEEN 7 AND
12 THEN 'Morning'
           WHEN EXTRACT(hour from order_purchase_timestamp) BETWEEN 12 AND
18 THEN 'Afternoon'
           WHEN EXTRACT(hour from order_purchase_timestamp) BETWEEN 19 AND
23 THEN 'Night'
      END AS Time_ordered,
      count(order_id) as No_of_orders
FROM
      TARGET.orders
GROUP BY 1
ORDER BY 2 DESC;
```

output:

Row	Time_ordered	No_of_orders
1	Afternoon	38135
2	Night	28331
3	Morning	27733
4	Dawn	5242

Insights and recommendations:



- The **Afternoon** time slot receives the highest number of orders: **38,135 (38.3%)**, indicating peak user activity during this period.
- **Night (28,331 | 28.5%)** and **Morning (27,733 | 27.9%)** contribute nearly equally, showing consistent engagement throughout the day.
- **Dawn** has significantly fewer orders: **5,242 (5.3%)**, making it the least active slot.

Recommendation: *To maximize ROI, promotional campaigns and advertising should be focused during the **Afternoon**, when user activity peaks, while **Night** and **Morning** can benefit from targeted offers. Activity during **Dawn** is minimal and may not justify major campaigns, though niche strategies can still be explored.*

3. Evolution of E-commerce orders in the Brazil region

3.1 Get the month on month no. of orders placed in each state.

Query:

```
SELECT
      c.customer_state as State,
      FORMAT_DATE('%Y-%m', o.order_purchase_timestamp) as Month,
      count(o.order_id) as No_of_orders
FROM
      TARGET.orders as o
JOIN  TARGET.customers as c
  ON o.customer_id = c.customer_id
GROUP BY 1,2
ORDER BY state, month;
```

output:

Row	State	Month	No_of_orders
1	AC	2017-01	2
2	AC	2017-02	3
3	AC	2017-03	2
4	AC	2017-04	5
5	AC	2017-05	8
6	AC	2017-06	4
7	AC	2017-07	5
8	AC	2017-08	4
9	AC	2017-09	5
10	AC	2017-10	6

Insights and recommendations:

- **SP, MG & RJ** consistently lead in MoM orders, all hitting their highest volumes in **Nov 2017** and maintaining strong levels through mid-2018.

- Most other states follow a similar late-year peak, indicating a nationwide seasonal effect.
- **AC, AP & RR** remain at the bottom with sporadic, low order counts, highlighting regional gaps.

Recommendation: *Adequate stock, marketing, and customer support should be secured in SP, MG, and RJ ahead of the October–November peak, while major Q4 promotions launched nationwide can leverage the year-end surge, and targeted social-media ads and localized discounts in AC, AP, and RR can stimulate growth in these underperforming regions.*

3.2 How are the customers distributed across all the states?

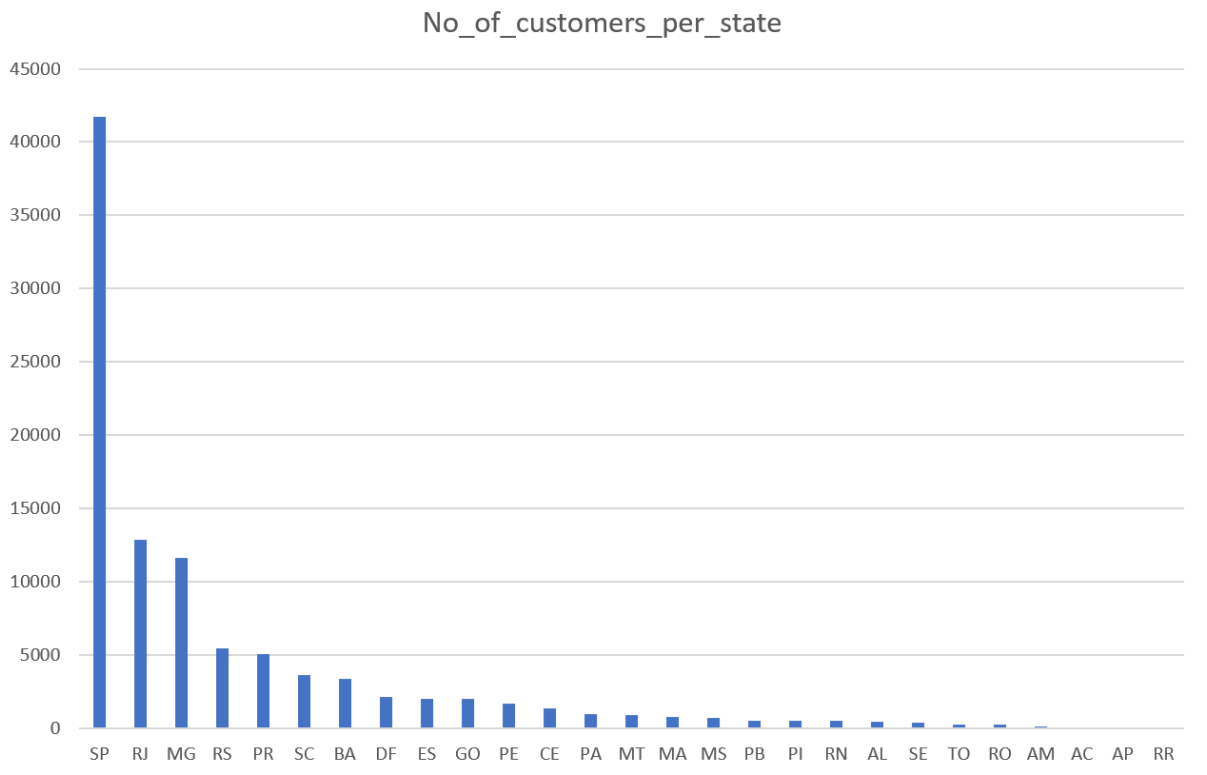
Query:

```
SELECT
      customer_state as State,
      count(distinct customer_id) as No_of_customers
FROM
      `TARGET.customers`
GROUP BY  customer_state
ORDER BY no_of_customers DESC;
```

output:

Row	State	No_of_customers
1	SP	41746
2	RJ	12852
3	MG	11635
4	RS	5466
5	PR	5045
6	SC	3637
7	BA	3380
8	DF	2140
9	ES	2033
10	GO	2020

Insights and recommendations:



- **SP leads** with **41,746 customers**, followed by **RJ (12,852)** and **MG (11,635).** A mid-tier group—including RS (5,466), PR (5,045), SC (3,637), BA (3,380), DF (2,140), ES (2,033), and GO (2,020)—accounts for moderate volumes, while states like **RR, PI, and RO register fewer than 1,000** customers each.

Recommendation: *Concentrate engagement efforts in SP, RJ, and MG to sustain their large user bases, tailor awareness and retention campaigns for mid-tier states (RS, PR, SC), and deploy targeted acquisition initiatives in underrepresented markets such as RR, PI, and RO to broaden overall reach.*

4. Impact on Economy: Analyse the money movement by e-commerce by looking at order prices, freight and others.

4.1 Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).

Query:

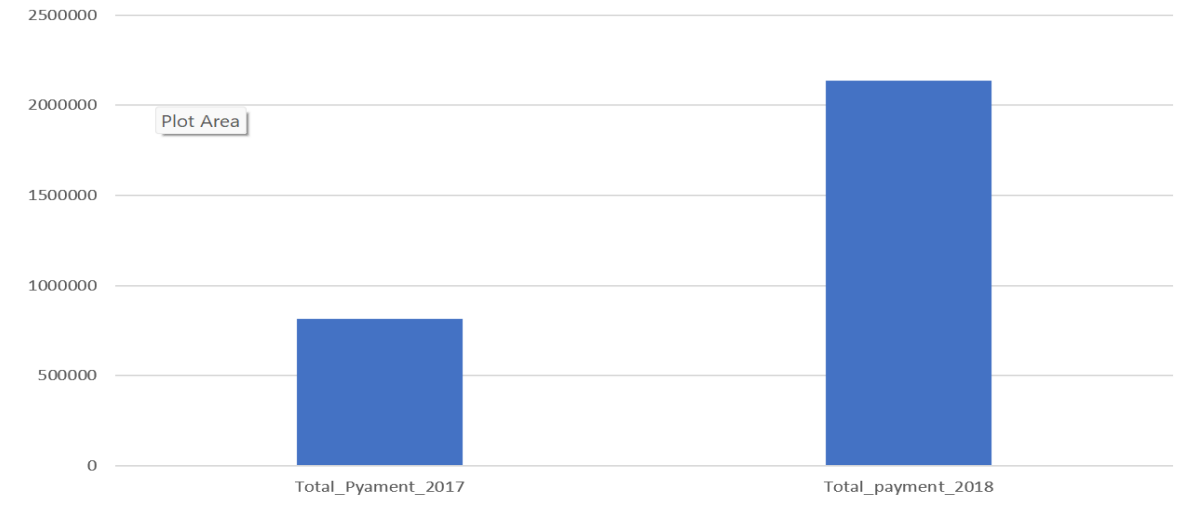
```
WITH CTE AS(SELECT
                EXTRACT(year FROM o.order_purchase_timestamp) AS Year,
```

```
                SUM(p.payment_value) AS Total_payment_value

            FROM
                `TARGET.orders` AS o
            JOIN
                TARGET.payments AS p
              ON o.order_id = p.order_id
            WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) IN (2017,
2018)
            AND   EXTRACT(MONTH FROM o.order_purchase_timestamp)IN (1,8)
            GROUP BY 1)


SELECT
    round(p17.pv) as Total_Pyament_2017,
    round(p18.pv) as Total_payment_2018,
    round(p18.pv/(p17.pv + p18.pv) * 100) as YoY_change

FROM (SELECT
          total_payment_value as pv
     FROM
          cte
      WHERE year = 2017) AS p17,
      (SELECT
          total_payment_value as pv
     FROM
          cte
      WHERE year = 2018) AS p18;
```

output:

Row	Total_Pyament_2017	Total_payment_2018	YoY_change
1	812884.0	2137430.0	72.0

Insights and recommendations:



- Between January–August 2017 and the same period in 2018, total payment value jumped from **812,884** to **2,137,430**, representing a **72% year-over-year increase**. This sharp rise indicates strong growth in order volume and/or average order value.

4.2 Calculate the Total & Average value of order price for each state.

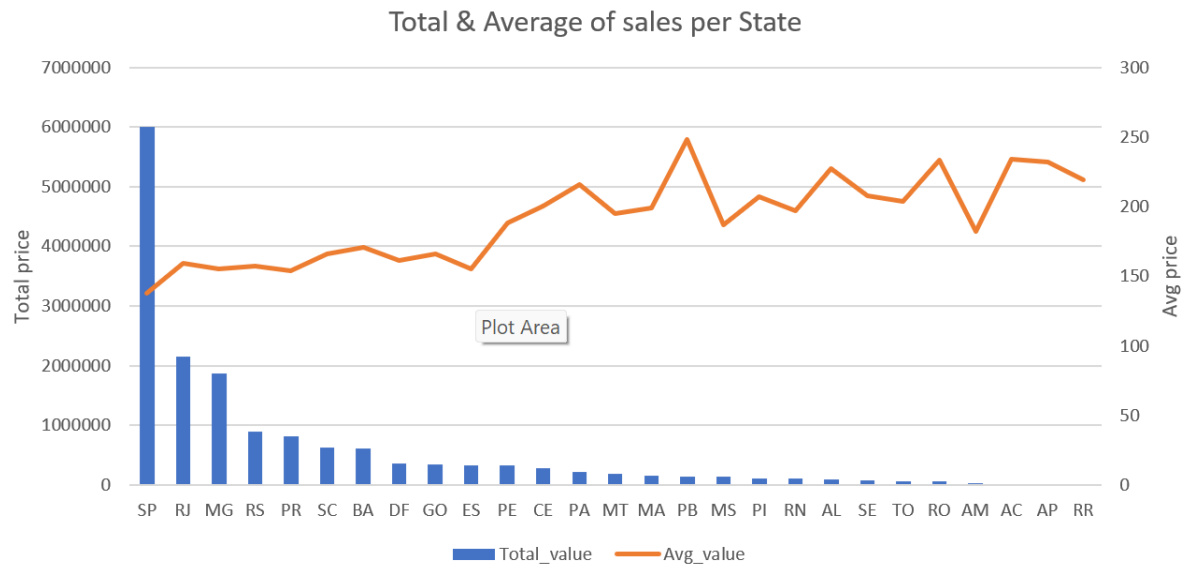
Query:

```
SELECT
  c.customer_state,
  ROUND(SUM(p.payment_value)) as Total_value,
  ROUND(AVG(p.payment_value)) as Avg_value
FROM
   `TARGET.payments` as p
JOIN
  `TARGET.orders` as o
  on p.order_id = o.order_id
JOIN
  `TARGET.customers` as c
  on o.customer_id = c.customer_id
GROUP BY c.customer_state
ORDER BY total_value DESC, avg_value DESC;
```

output:

Row	customer_state	Total_value	Avg_value
1	SP	5998227.0	138.0
2	RJ	2144380.0	159.0
3	MG	1872257.0	155.0
4	RS	890899.0	157.0
5	PR	811156.0	154.0
6	SC	623086.0	166.0
7	BA	616646.0	171.0
8	DF	355141.0	161.0
9	GO	350092.0	166.0
10	ES	325968.0	155.0

Insights and recommendations:



- **SP** continues to dominate with the highest total order value of **5,998,227**, though its **average order value remains low at 138**, signalling a **high transaction volume but low per-order spend**. On the other hand, states like **PB (avg: 248), AC (234), RO (233), AP (232), and RR (219)** show **exceptionally high average order values** despite having **low total values**, indicating **niche but high-value customer segments**. **BA (171)** and **SC (166)** maintain a **strong mid-tier presence**, combining moderate totals with healthy averages. States such as **RJ, MG, RS, and PR** reflect a **balanced purchasing pattern** with decent totals and averages around **154–159**.

Recommendation: *Focus on driving up the average order value in SP through bundled product offers or value-based promotions. Simultaneously, retain high-value customers in states like PB, AC, and RO by ensuring premium product availability and personalized service. Maintain steady growth in RJ, MG, and PR with consistent marketing efforts and optimize inventory and pricing strategy in middle-performing states like BA and SC to sustain their profitability.*

4.3 Calculate the Total & Average value of order freight for each state.

Query:

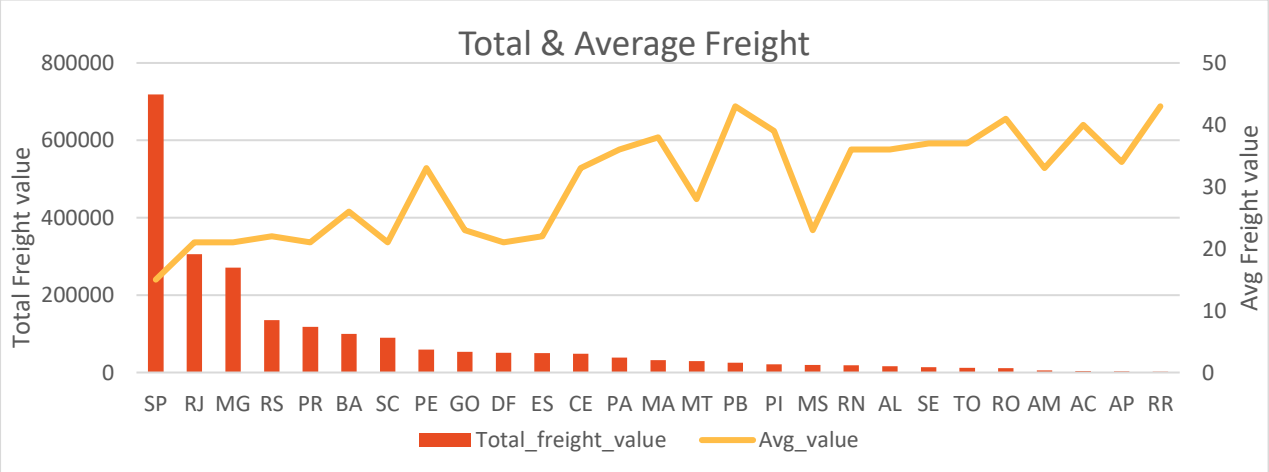
```
SELECT
```

```sql
  c.customer_state,
  ROUND(SUM(p.freight_value)) as Total_freight_value,
  ROUND(AVG(p.freight_value)) as Avg_value
FROM
   `TARGET.order_items` as p
JOIN
  `TARGET.orders` as o
  on p.order_id = o.order_id
JOIN
  `TARGET.customers` as c
  on o.customer_id = c.customer_id
GROUP BY c.customer_state
ORDER BY total_freight_value DESC, avg_value DESC;
```

output:

Row	customer_state	Total_freight_value	Avg_value
1	SP	718723.0	15.0
2	RJ	305589.0	21.0
3	MG	270853.0	21.0
4	RS	135523.0	22.0
5	PR	117852.0	21.0
6	BA	100157.0	26.0
7	SC	89660.0	21.0
8	PE	59450.0	33.0
9	GO	53115.0	23.0
10	DF	50626.0	21.0

Insights and recommendations:



- **SP has the highest total freight value with a low average cost (15)**, suggesting a high order volume. States like **RR and PB show the highest average freight** costs (43) despite having low total freight values, **indicating higher delivery expenses per order.**

Recommendation: *Focus on improving delivery efficiency in high-cost states such as RR and PB. In SP, continue leveraging economies of scale to maintain low freight costs despite high demand.*

5. Analysis based on sales, freight and delivery time.

5.1 Find the no. of days taken to deliver each order from the order's purchase date as delivery time. Also, calculate the difference (in days) between the estimated & actual delivery date of an order.
Do this in a single query.

Query:

```
SELECT
    order_id,
    DATE_DIFF(DATE(order_delivered_customer_date), DATE
(order_purchase_timestamp), DAY) AS Delivery_Time,
    DATE_DIFF(DATE(order_delivered_customer_date),
DATE(order_estimated_delivery_date), DAY) AS
Acutal_date_diff_from_estimated_del
FROM
    `TARGET.orders`
WHERE order_status = 'delivered';
```

output:

Row	order_id	Delivery_Time	Acutal_date_diff_...
1	bfbd0f9bdef84302105ad712db...	55	36
2	98974b076b01553d49ee64679...	44	-7
3	c4b41c36dd589e901f6879f25a...	36	-15
4	d2292ff2201e74c5db154d1b7a...	30	-21
5	95e01270fcbae986342340010...	31	-20
6	ed8c7b1b3eb256c70ce0c7423...	45	-6
7	5cc475c7c03290048eb2e742c...	69	18
8	6b3ee7697a02619a0ace2b3f0a...	48	-3
9	3b2ca3293a7ce539ea2379d70...	43	-8
10	b2f92b2f7047cd8b35580d629d...	43	-8

Insights and recommendations:

- Used aggregate functions like MAX, MIN, and AVG, it was observed that the **average delivery time** is approximately **12.5 days**, with some orders delivered **on the same day** and others taking up to **210 days**. The **average deviation** from the estimated delivery date is around **-11.88 days**, showing that most orders arrive **earlier than expected**, though some were delivered up to **188 days late** or **147 days early**, reflecting inconsistency in delivery timelines.

Recommendation: *Improve the **accuracy of estimated delivery dates** to better manage customer expectations. Analyse cases with extreme delays or unusually early deliveries to identify bottlenecks or best practices. Standardizing these learnings could help reduce variability and enhance overall delivery performance.*

5.2 Find out the top 5 states with the highest & lowest average freight value.

Query:

```
WITH fr AS (
  SELECT
    c.customer_state AS State,
    AVG(p.freight_value) AS avg_freight,
    DENSE_RANK() OVER(ORDER BY AVG(p.freight_value) DESC) AS Freight_rank_asc,
    DENSE_RANK() OVER(ORDER BY AVG(p.freight_value)) AS Freight_rank_desc
  FROM `TARGET.order_items` AS p
  JOIN `TARGET.orders` AS o ON p.order_id = o.order_id
  JOIN `TARGET.customers` AS c ON o.customer_id = c.customer_id
  GROUP BY c.customer_state
),
Top_states AS (
  SELECT
    State AS Highest_freight_state,
    ROW_NUMBER() OVER () AS rn
  FROM fr
  WHERE Freight_rank_asc <= 5
),
Low_states AS (
  SELECT
    State AS Lowest_freight_state,
    ROW_NUMBER() OVER () AS rn
  FROM fr
  WHERE Freight_rank_desc <= 5
)
SELECT
  t.Highest_freight_state,
  l.Lowest_freight_state
FROM Top_states t
FULL OUTER JOIN Low_states l
```

```
ON t.rn = l.rn;
```

output:

Row	Highest_freight_state	Lowest_freight_state
1	AC	DF
2	PI	SP
3	RR	MG
4	RO	PR
5	PB	RJ

Insights and recommendations:

- The **highest average freight costs occur in AC, PI, RR, RO, and PB**—states that likely face higher shipping expenses—while **DF, SP, MG, PR, and RJ enjoy the lowest freight rates**, reflecting their proximity to major distribution centres and efficient logistics.

Recommendation: *Introduce regional fulfilment hubs or optimized shipping routes in AC, PI, RR, RO, and PB to bring down per-order freight costs, and apply the best practices from DF, SP, MG, PR, and RJ across other regions to improve overall shipping efficiency.*

5.3 Find out the top 5 states with the highest & lowest average delivery time.

Query:

```
WITH state_avg AS (
  SELECT
    c.customer_state AS state,
    AVG(
      DATE_DIFF(
        DATE(order_delivered_customer_date),
```

```sql
        DATE(order_purchase_timestamp),
        DAY
      )
    ) AS avg_delivery_time
  FROM
    `TARGET.orders` o
  JOIN
    `TARGET.customers` c
      ON o.customer_id = c.customer_id
  WHERE
    o.order_status = 'delivered'
  GROUP BY
    state
),

top5 AS (
  SELECT
    state        AS highest_avg_delivery_state,
    ROW_NUMBER() OVER (ORDER BY avg_delivery_time DESC) AS rn
  FROM
    state_avg
  ORDER BY
    avg_delivery_time DESC
  LIMIT 5
),

low5 AS (
  SELECT
    state        AS lowest_avg_delivery_state,
    ROW_NUMBER() OVER (ORDER BY avg_delivery_time ASC) AS rn
  FROM
    state_avg
  ORDER BY
    avg_delivery_time ASC
  LIMIT 5
)

SELECT
  t.highest_avg_delivery_state,
  l.lowest_avg_delivery_state
FROM
  top5 t
FULL OUTER JOIN
  low5 l
ON
  t.rn = l.rn;
```

output:

Row	highest_avg_delivery_state	lowest_avg_delivery_state
1	AP	PR
2	AM	MG
3	RR	SP
4	AL	DF
5	PA	SC

Insights and recommendations:

- The states **AP**, **AM**, **RR**, **AL**, and **PA** have the **highest average delivery times**, indicating persistent logistical challenges or distance-related delays in those regions. In contrast, **PR**, **MG**, **SP**, **DF**, and **SC** achieve the **fastest deliveries**, reflecting their proximity to major fulfilment centres and more efficient shipping networks.

Recommendation: *To reduce delivery times in the slower states (AP, AM, RR, AL, PA), consider establishing **regional fulfilment hubs** or **micro-warehouses** closer to these markets and partnering with local carriers to streamline last-mile delivery. Meanwhile, codify and replicate the optimized processes used in the fastest states (PR, MG, SP, DF, SC)—such as centralized sorting facilities and prioritized routing—to improve consistency across all regions*.

5.4 Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.

Query:

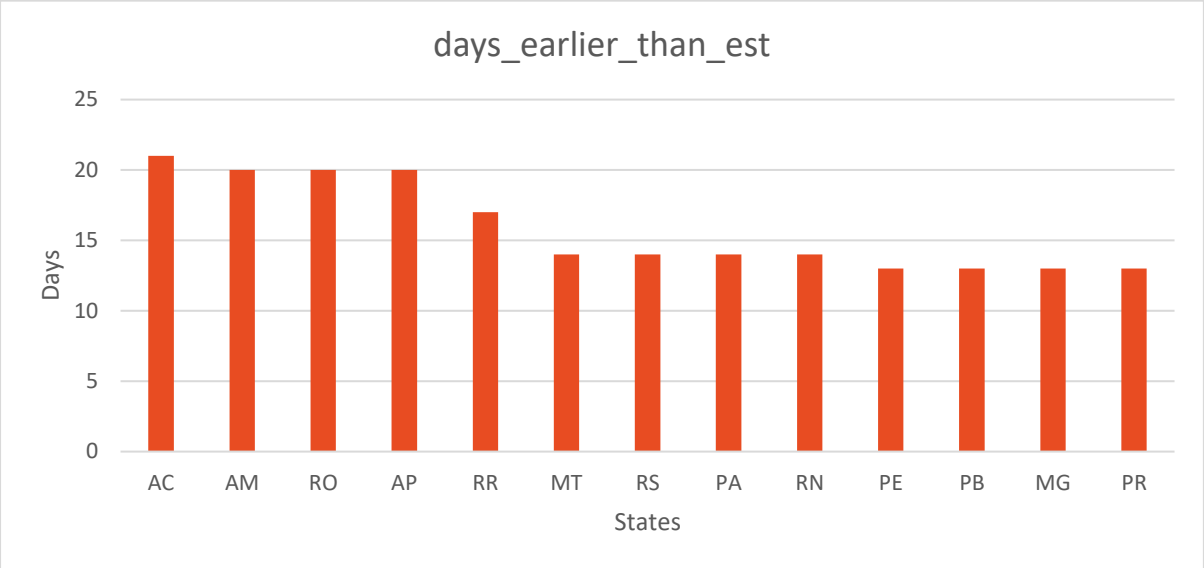
```
WITH temp AS (
  SELECT
    c.customer_state AS State,
    ROUND(
      AVG(
        DATE_DIFF(
          DATE(o.order_estimated_delivery_date),
          DATE(o.order_delivered_customer_date),
          DAY
        )
      )
    ) AS days_earlier_than_est
```

```
  FROM `TARGET.orders` o
  JOIN `TARGET.customers` c
    ON o.customer_id = c.customer_id
  WHERE o.order_status = 'delivered'
  GROUP BY State
)

SELECT
  State,
  days_earlier_than_est
FROM (
  SELECT
    State,
    days_earlier_than_est,
    DENSE_RANK() OVER (ORDER BY days_earlier_than_est DESC) AS rk
  FROM temp
)
WHERE rk <= 5
ORDER BY days_earlier_than_est DESC;
```

output:

Row	State	days_earlier_than_est
1	AC	21.0
2	AM	20.0
3	RO	20.0
4	AP	20.0
5	RR	17.0
6	MT	14.0
7	RS	14.0
8	PA	14.0
9	RN	14.0
10	PE	13.0
11	PB	13.0
12	MG	13.0
13	PR	13.0

Insights and recommendations:



- The states **AC** (21 days early), **AM** (20), **RO** (20), **AP** (20), and **RR** (17) deliver far ahead of their estimates—over two to three weeks early—while a second tier (MT, RS, PA, RN) averages 14 days early and the rest around 13 days early.

Recommendation: *Investigate the expedited fulfilment processes used in AC, AM, RO, AP, and RR—such as proximity to micro-warehouses or priority carrier partnerships and apply those best practices to other regions to consistently exceed delivery expectations.*

6. Analysis based on the payments

6.1 Find the month-on-month no. of orders placed using different payment types.

Query:

```
SELECT
    FORMAT_DATE('%Y-%m', o.order_purchase_timestamp) as Month,
    p.payment_type,
    COUNT(o.order_id) as No_of_orders
FROM
  `TARGET.orders` as o
JOIN
  `TARGET.payments` as p
```

```
 ON o.order_id = p.order_id
GROUP BY 1,2
ORDER BY month ,no_of_orders DESC;
```

output:

Row	Month	payment_type	No_of_orders
1	2016-09	credit_card	3
2	2016-10	credit_card	254
3	2016-10	UPI	63
4	2016-10	voucher	23
5	2016-10	debit_card	2
6	2016-12	credit_card	1
7	2017-01	credit_card	583
8	2017-01	UPI	197
9	2017-01	voucher	61
10	2017-01	debit_card	9

Insights and recommendations:

- **Credit cards overwhelmingly dominate monthly orders—peaking at 5,691 orders in March 2018**—while **UPI consistently ranks second** (around 1,000–1,500 orders/month). Voucher purchases hold a modest share (~300 orders/month), and **debit-card transactions remain minimal**.

Recommendation: *Streamline and promote alternative digital methods—particularly UPI and debit-card payments—through targeted incentives or simplified checkout flows to diversify payment mix, while ensuring every transaction is properly logged to eliminate the "not_define" category.*

6.2 Find the no. of orders placed on the basis of the payment instalments that have been paid.

Query:

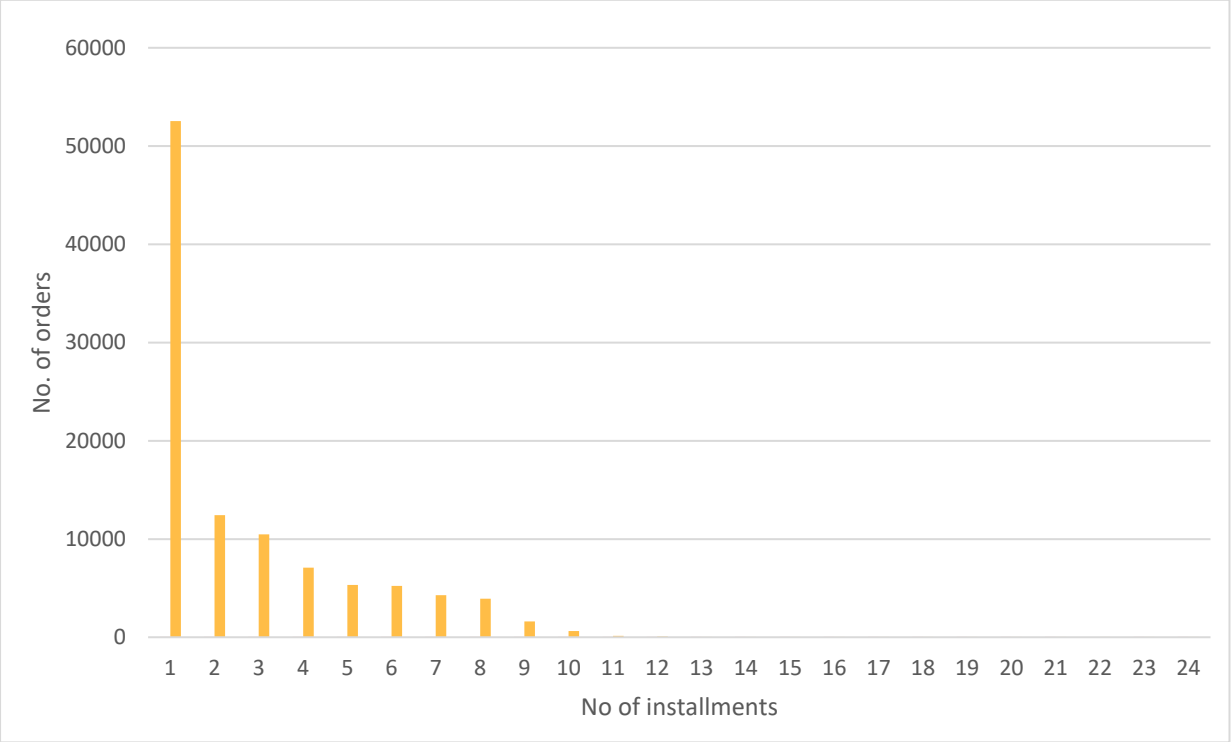
```
SELECT
```

```
  p.payment_installments,
  COUNT(o.order_id) as No_of_orders
FROM
  `TARGET.orders` as o
JOIN
  `TARGET.payments` as p
 ON o.order_id = p.order_id
GROUP BY p.payment_installments
ORDER BY no_of_orders  DESC;
```

output:

Row	payment_installments	No_of_orders
1	1	52546
2	2	12413
3	3	10461
4	4	7098
5	10	5328
6	5	5239
7	8	4268
8	6	3920
9	7	1626
10	9	644

Insights and recommendations:



- **Single-instalment payments dominate**, with **52,546 orders** paid in 1 instalment—about **60%** of all transactions.
- **Two- and three-instalment plans** are the next most popular, with **12,413** and **10,461** orders respectively.
- Beyond four instalments, usage drops off sharply: fewer than **7,098** for 4-installment plans and under **5,328** for 10-installment plans.

Recommendation: *Focus promotional efforts on **1–3 instalment options**, which account for the vast majority of orders—consider bundling small discounts or zero-interest offers on 2- and 3-month EMIs to shift some 1-installment volume into multi-payment plans, thereby increasing average order value and spreading revenue over time.*